Domain 4 — Data Loading, Unloading & Connectivity

Complete Beginner's Master Guide | COF-C03 2026

Technical + Layman Definitions | All Subdomains | Edge Cases | Hands-On SQL | 20 Questions Per Topic | Full Answer Key

> How to use this guide
>
> 1. Read **Layman Explanation** first — build the mental model
> 2. Read **Technical Explanation** — precise mechanics, every edge case, exam traps
> 3. Run every **Hands-On Query block** in your Snowflake free trial
> 4. Attempt all 20 questions per topic WITHOUT peeking at answers
> 5. Check the annotated **Answer Key** at the very end

> **Exam facts:** Domain 4 (Data Loading, Unloading & Connectivity) = **18% of the COF-C03 exam.** It covers how data gets in, how it gets out, and how tools connect to Snowflake.[1][2]

SUBDOMAIN 4.1 — Stages, File Formats & COPY INTO

Layman's Explanation

Before you can bring data into Snowflake, you need a **staging area** — think of it like a receiving dock at a warehouse. You drop files off at the dock, inspect them, and then move them into the actual storage shelves (Snowflake tables).

Snowflake has three kinds of docks:

- **Internal Stages:** Docks INSIDE Snowflake's own facility — you upload files directly into Snowflake's managed storage
- **External Stages:** Docks at YOUR cloud storage (S3, Azure Blob, Google Cloud Storage) — Snowflake reaches out and picks up your files
- **User/Table Stages:** Personal mini-docks automatically assigned to every user and every table — no setup required

Once the files are staged, **COPY INTO** is the forklift that physically moves data from the dock into the table.

A **File Format** tells COPY INTO how to "read" the files — are they comma-separated? JSON? Compressed? What character is the field delimiter?

⊠ Technical Explanation

Stage Types — Exhaustive Guide[3][1]

Stage Type	Name Syntax	Storage Location	Created By	Use Case
User Stage	`@~`	Snowflake-managed (per user)	Auto-created per user	Per-user temporary file staging; cannot be shared
Table Stage	`@%table_name`	Snowflake-managed (per table)	Auto-created per table	Loading data into a specific table; accessible only to that table's owners
Named Internal Stage	`@my_stage`	Snowflake-managed, named object	CREATE STAGE	Shared staging area; supports multiple files, directories
Named External Stage	`@my_ext_stage`	Your S3/Azure/GCS bucket	CREATE STAGE... URL=	Load from your existing cloud storage; files remain in your cloud

Critical edge cases:

- User stages and Table stages **cannot have File Format objects assigned** at the stage level — you specify the format in the COPY INTO command or `@~` notation
- User Stage does NOT support Directory Tables
- Only Named Stages can be shared across roles and users
- Only Named External Stages require a **Storage Integration** (or direct credential embedding, which is less secure)

Storage Integrations — Secure External Access[4][3]

What it is: A Snowflake account-level object that stores a trusted relationship between Snowflake's IAM identity and your cloud provider account. This allows Snowflake to access external storage WITHOUT embedding access keys in the stage definition.

Why it matters (exam favorite): Storage Integrations avoid exposing cloud credentials in stage DDL — safer and easier to rotate.

```
-- Step 1: Create the storage integration (ACCOUNTADMIN required)
CREATE STORAGE INTEGRATION my_s3_integration
    TYPE = EXTERNAL_STAGE
    STORAGE_PROVIDER = 'S3'
    ENABLED = TRUE
    STORAGE_ALLOWED_LOCATIONS = ('s3://my-bucket/data/', 's3://my-other-bucket/')
```

```
    -- STORAGE_BLOCKED_LOCATIONS can restrict sub-paths
;

-- Step 2: Retrieve the IAM details to grant to Snowflake in AWS
DESC INTEGRATION my_s3_integration;
-- Copy: STORAGE_AWS_IAM_USER_ARN and STORAGE_AWS_EXTERNAL_ID
-- Go to AWS → IAM → attach these to your S3 bucket trust policy

-- Step 3: Create a stage using the integration (no credentials needed!)
CREATE STAGE my_secure_external_stage
    URL = 's3://my-bucket/data/'
    STORAGE_INTEGRATION = my_s3_integration
    FILE_FORMAT = (TYPE = 'CSV' FIELD_DELIMITER = ',' SKIP_HEADER = 1);
```

File Formats — Complete Reference[1][3]

Named File Format vs. **Inline Format**:

- **Named File Format:** A reusable schema-level object created with `CREATE FILE FORMAT` — can be assigned to stages or used in COPY INTO
- **Inline Format:** File format options specified directly inside the COPY INTO statement — not reusable

CSV (Delimited) — Key Parameters

Parameter	Default	Description
`FIELD_DELIMITER`	`,`	Column separator character
`RECORD_DELIMITER`	`\n`	Row separator
`SKIP_HEADER`	0	Lines to skip at file start (for header rows)
`NULL_IF`	`('\\N', 'NULL', '')`	Strings treated as NULL
`EMPTY_FIELD_AS_NULL`	TRUE	Empty fields become NULL
`FIELD_OPTIONALLY_ENCLOSED_BY`	NONE	Quote character around fields (usually `'"'`)
`ESCAPE`	NONE	Escape character for special chars in values
`DATE_FORMAT`	AUTO	Date parsing format
`TIMESTAMP_FORMAT`	AUTO	Timestamp parsing format
`ENCODING`	UTF8	Character encoding
`TRIM_SPACE`	FALSE	Strip whitespace from fields
`ERROR_ON_COLUMN_COUNT_MISMATCH`	TRUE	Error if column count in file ≠ table columns

Parameter	Default	Description
COMPRESSION	AUTO	AUTO detects: gzip, bzip2, brotli, zstd, deflate, raw_deflate

JSON — Key Parameters

Parameter	Default	Description
STRIP_OUTER_ARRAY	FALSE	Remove the outer JSON array [...] — must be TRUE when loading array of objects
STRIP_NULL_VALUES	FALSE	Remove NULL value key-value pairs from objects
IGNORE_UTF8_ERRORS	FALSE	Replace malformed UTF-8 characters
ALLOW_DUPLICATE	FALSE	Allow duplicate keys in JSON objects

Critical exam trap: When loading a JSON file that is a single array of objects (`[{...},{...}]`), you MUST set `STRIP_OUTER_ARRAY = TRUE` — otherwise the entire array is loaded as a single VARIANT row.

Parquet / ORC / Avro — Key Parameters

- `SNAPPY_COMPRESSION` — default compression for Parquet
- `MATCH_BY_COLUMN_NAME` — for Parquet: match file columns to table columns by NAME instead of position (important when column order differs)
- Parquet files are columnar by nature — efficient for analytics

XML

- `STRIP_OUTER_ELEMENT` — similar to STRIP_OUTER_ARRAY for XML documents
- `DISABLE_AUTO_CONVERT` — disable automatic type conversion

Directory Tables[4][3]

What they are: A built-in table-like view available on NAMED STAGES (internal and external) that lists all files staged there — metadata including file path, file size, last modified date, MD5 checksum.

```
-- Enable directory table on a stage
ALTER STAGE my_stage REFRESH;  -- Sync the directory metadata

-- Query the directory table
SELECT * FROM DIRECTORY(@my_stage);
-- Returns: relative_path, file_url, last_modified, file_size, etag, file_md5

-- List files in a stage (alternative to directory table)
```

```
LIST @my_stage;
LIST @my_stage PATTERN = '.*sales.*';  -- Filter by filename pattern
```

Edge cases:

- `ALTER STAGE ... REFRESH` must be run to sync metadata after files are added/removed externally
- Auto-refresh can be enabled for external stages connected to event notifications (S3 Event, Azure Event Grid)
- Directory Tables are NOT available on User Stages (`@~`) or Table Stages (`@%table`)

COPY INTO (Loading) — Complete Reference[3][1]

Basic syntax:

```
COPY INTO target_table
FROM @stage_or_location
FILE_FORMAT = (FORMAT_NAME = 'my_csv_format' | TYPE = 'CSV' ...)
FILES = ('file1.csv', 'file2.csv')       -- specific files (optional)
PATTERN = '.*sales_2025.*\\.csv'         -- regex pattern (optional)
ON_ERROR = ABORT_STATEMENT               -- error handling behavior
FORCE = FALSE                           -- re-load already-loaded files?
PURGE = FALSE                           -- delete staged files after load?
MATCH_BY_COLUMN_NAME = NONE              -- column matching for Parquet/ORC/Avro
VALIDATION_MODE = NULL                  -- validate without loading
;
```

ON_ERROR Options — Full Breakdown (Exam Favorite)[1][3]

Option	Behavior	When to Use
`ABORT_STATEMENT` (default)	Stop entire COPY operation at first error. No rows loaded. Entire operation rolled back.	Data quality is critical — all-or-nothing
`CONTINUE`	Skip error rows, continue loading. All valid rows loaded.	Load as much as possible; bad rows can be reviewed in VALIDATE() later
`SKIP_FILE`	Skip the ENTIRE file containing any error. Move to next file.	File-level granularity; don't want partial file loads
`SKIP_FILE_num` (e.g., `SKIP_FILE_10`)	Skip the file if it has MORE THAN num error rows	Skip files with excessive errors, load files with minor issues
`SKIP_FILE_num%` (e.g., `SKIP_FILE_5%`)	Skip the file if error rate EXCEEDS num% of total rows	Percentage-based tolerance

FORCE = TRUE: Re-load files already loaded (bypasses 64-day load history deduplication). Use with caution — can cause duplicates.

PURGE = TRUE: Delete source files from the stage after successful load. Not available for User Stages.

VALIDATION_MODE — Load Without Actually Loading[3][1]

```
-- Validate file without loading — returns rows that WOULD fail
COPY INTO my_table
FROM @my_stage
FILE_FORMAT = (FORMAT_NAME = 'my_format')
VALIDATION_MODE = RETURN_ERRORS;
-- Returns: all rows that would error, with error details

COPY INTO my_table
FROM @my_stage
VALIDATION_MODE = RETURN_ALL_ERRORS;
-- Returns all errors (not just first 500)

COPY INTO my_table
FROM @my_stage
VALIDATION_MODE = RETURN_N_ROWS(100);
-- Returns first 100 rows that WOULD be loaded (preview)

-- After a COPY INTO run, review errors
SELECT * FROM TABLE(VALIDATE(my_table, JOB_ID => '_last'));
-- Returns all errors from the last COPY INTO operation
```

64-Day Load History[1][3]

What it is: Snowflake tracks every file loaded via COPY INTO for 64 days per table. If you try to COPY a file already loaded into the same table within 64 days, Snowflake SKIPS it (deduplication).

Key rules:

- File tracking is **per table** — same file can be loaded into DIFFERENT tables without being skipped
- `FORCE = TRUE` overrides the 64-day check — forces re-load regardless of history
- Managed via `LOAD_HISTORY` view: `SELECT * FROM SNOWFLAKE.ACCOUNT_USAGE.LOAD_HISTORY`

COPY INTO (Unloading — Data Export)[3][1]

```
-- Unload table to internal stage
COPY INTO @my_stage/unload_folder/
FROM (SELECT * FROM my_table WHERE region = 'North')
FILE_FORMAT = (TYPE = 'CSV' COMPRESSION = 'GZIP')
SINGLE = FALSE                          -- Multiple files (parallel, faster)
MAX_FILE_SIZE = 52428800                -- Max 50MB per file
HEADER = TRUE                          -- Include column headers in CSV
```

```
OVERWRITE = TRUE;                       -- Overwrite existing files in stage

-- Unload to external stage
COPY INTO @my_external_stage/exports/
FROM my_table
FILE_FORMAT = (TYPE = 'PARQUET')
SINGLE = FALSE;
```

SINGLE parameter:

- `SINGLE = FALSE` (default): Multiple parallel files — faster, but no single output file
- `SINGLE = TRUE`: One file — slower, no parallelism, useful when downstream system expects one file

HEADER parameter: Only valid for CSV format. Adds column names as first row.

Hands-On Queries: Stages, File Formats & COPY INTO

```
-- ============================================================
-- SETUP
-- ============================================================
USE ROLE SYSADMIN;
CREATE DATABASE IF NOT EXISTS LOADING_LAB;
USE DATABASE LOADING_LAB;
CREATE SCHEMA IF NOT EXISTS STAGE_LAB;
USE SCHEMA STAGE_LAB;
CREATE WAREHOUSE IF NOT EXISTS load_wh WAREHOUSE_SIZE = 'X-SMALL' AUTO_SUSPEND = 60 A
USE WAREHOUSE load_wh;

-- ============================================================
-- PART 1: Internal Stages
-- ============================================================
-- User Stage (automatically exists for every user)
-- PUT command uploads from local machine (use SnowSQL CLI or Snowsight)
-- PUT file://~/data/sales.csv @~;  -- Upload from local file system (not in web UI)

-- List User Stage contents
LIST @~;

-- Table Stage (automatically exists for every table)
CREATE OR REPLACE TABLE employees (
    emp_id    INTEGER,
    name      VARCHAR(100),
    dept      VARCHAR(50),
    salary    NUMBER(12,2),
    hire_date DATE
);

-- List Table Stage for employees table
LIST @%employees;

-- Named Internal Stage
CREATE OR REPLACE STAGE my_internal_stage
```

```
    COMMENT = 'Named internal stage for loading employee data';

-- With a file format assigned at stage level
CREATE OR REPLACE FILE FORMAT my_csv_format
    TYPE = 'CSV'
    FIELD_DELIMITER = ','
    SKIP_HEADER = 1
    NULL_IF = ('NULL', 'null', '')
    EMPTY_FIELD_AS_NULL = TRUE
    FIELD_OPTIONALLY_ENCLOSED_BY = '"'
    DATE_FORMAT = 'YYYY-MM-DD'
    COMPRESSION = 'AUTO';

CREATE OR REPLACE STAGE my_formatted_stage
    FILE_FORMAT = my_csv_format
    COMMENT = 'Named internal stage with pre-assigned CSV format';

-- List stage contents
LIST @my_internal_stage;
LIST @my_formatted_stage;

-- ============================================================
-- PART 2: Working with CSV Data Using $COPY
-- Use Snowflake INSERT to simulate staged files:
-- ============================================================
-- Create an employees table and simulate a "load" using VALUES
INSERT INTO employees VALUES
    (1, 'Alice Johnson', 'Engineering', 95000.00, '2020-03-15'),
    (2, 'Bob Smith',     'Marketing',  72000.00, '2019-11-01'),
    (3, 'Carol Lee',     'Finance',    88000.00, '2021-06-30'),
    (4, 'David Chen',    'Engineering',102000.00, '2018-08-20'),
    (5, 'Eva Martinez',  'HR',          65000.00, '2022-01-10');

SELECT * FROM employees;

-- ============================================================
-- PART 3: Named External Stage (connect to cloud storage)
-- ============================================================
-- OPTION A: Using Storage Integration (recommended — no keys exposed)
-- First run DESC INTEGRATION after creating it to get AWS IAM details
-- CREATE STORAGE INTEGRATION my_s3_int
--     TYPE = EXTERNAL_STAGE
--     STORAGE_PROVIDER = 'S3'
--     ENABLED = TRUE
--     STORAGE_ALLOWED_LOCATIONS = ('s3://my-company-bucket/data/');

-- THEN create stage using the integration:
-- CREATE STAGE my_s3_stage
--     URL = 's3://my-company-bucket/data/'
--     STORAGE_INTEGRATION = my_s3_int
--     FILE_FORMAT = my_csv_format;

-- OPTION B: Inline credentials (NOT recommended for production — for learning only
-- CREATE STAGE my_s3_stage_creds
--     URL = 's3://my-bucket/data/'
--     CREDENTIALS = (AWS_KEY_ID = 'AKIAXXXXXX' AWS_SECRET_KEY = 'xxxxxxxx')
```

```
--     FILE_FORMAT = my_csv_format;

-- ============================================================
-- PART 4: File Format Examples
-- ============================================================
-- JSON File Format
CREATE OR REPLACE FILE FORMAT my_json_format
    TYPE = 'JSON'
    STRIP_OUTER_ARRAY = TRUE    -- CRITICAL: removes [ ] from array of objects
    STRIP_NULL_VALUES = FALSE
    IGNORE_UTF8_ERRORS = FALSE;

-- Parquet File Format
CREATE OR REPLACE FILE FORMAT my_parquet_format
    TYPE = 'PARQUET'
    SNAPPY_COMPRESSION = TRUE;

-- List all file formats
SHOW FILE FORMATS IN SCHEMA STAGE_LAB;

-- Describe a file format
DESC FILE FORMAT my_csv_format;

-- ============================================================
-- PART 5: COPY INTO Loading
-- ============================================================
-- Create a target table for CSV loading demo
CREATE OR REPLACE TABLE sales_load (
    sale_id     INTEGER,
    sale_date   DATE,
    region      VARCHAR(30),
    amount      NUMBER(12,2),
    notes       VARCHAR(500)
);

-- Generate some CSV data and stage it manually for demo
-- (In practice you'd PUT a real file; here we create sample data to stage)
-- For this lab, use INSERT to simulate data load:
INSERT INTO sales_load
SELECT
    SEQ4() + 1,
    DATEADD('day', -MOD(SEQ4(), 365), CURRENT_DATE()),
    CASE MOD(SEQ4(), 4) WHEN 0 THEN 'North' WHEN 1 THEN 'South' WHEN 2 THEN 'East' EL
    ROUND(UNIFORM(10.0, 5000.0, RANDOM()), 2),
    NULL
FROM TABLE(GENERATOR(ROWCOUNT => 1000));

-- Unload to internal stage first (creates CSV files we can then reload)
CREATE OR REPLACE STAGE reload_stage FILE_FORMAT = my_csv_format;

COPY INTO @reload_stage/sales_export/
FROM sales_load
FILE_FORMAT = (TYPE = 'CSV' COMPRESSION = 'NONE' HEADER = TRUE)
SINGLE = FALSE;

-- List the exported files
```

```
LIST @reload_stage;

-- Truncate the table (simulate fresh load)
TRUNCATE TABLE sales_load;

-- COPY INTO — load from stage with different ON_ERROR options
-- Default behavior (ABORT_STATEMENT)
COPY INTO sales_load
FROM @reload_stage/sales_export/
FILE_FORMAT = (TYPE = 'CSV' FIELD_DELIMITER = ',' SKIP_HEADER = 1)
ON_ERROR = ABORT_STATEMENT;

SELECT COUNT(*) FROM sales_load;  -- Should be 1000 rows

-- ============================================================
-- PART 6: FORCE, PURGE and VALIDATION_MODE
-- ============================================================
-- Try to re-COPY the same files (will be SKIPPED due to 64-day history)
COPY INTO sales_load
FROM @reload_stage/sales_export/
FILE_FORMAT = (TYPE = 'CSV' FIELD_DELIMITER = ',' SKIP_HEADER = 1);
-- STATUS = LOADED → files are SKIPPED (already loaded — 64-day history)

-- FORCE overrides the history — will cause duplicates!
COPY INTO sales_load
FROM @reload_stage/sales_export/
FILE_FORMAT = (TYPE = 'CSV' FIELD_DELIMITER = ',' SKIP_HEADER = 1)
FORCE = TRUE;
SELECT COUNT(*) FROM sales_load;  -- Now 2000 rows (duplicated!)
TRUNCATE TABLE sales_load;

-- VALIDATION_MODE: Preview without loading
COPY INTO sales_load
FROM @reload_stage/sales_export/
FILE_FORMAT = (TYPE = 'CSV' FIELD_DELIMITER = ',' SKIP_HEADER = 1)
VALIDATION_MODE = RETURN_5_ROWS;  -- Preview first 5 rows without loading
SELECT COUNT(*) FROM sales_load;  -- 0 rows — nothing was loaded!

-- ============================================================
-- PART 7: COPY INTO Unloading
-- ============================================================
CREATE OR REPLACE STAGE unload_stage;

-- Unload to multiple compressed CSV files (parallel, fast)
COPY INTO @unload_stage/full_export/
FROM sales_load
FILE_FORMAT = (TYPE = 'CSV' COMPRESSION = 'GZIP' HEADER = TRUE)
SINGLE = FALSE
MAX_FILE_SIZE = 104857600;   -- 100MB max per file

-- Unload to ONE single file (slower but single output)
COPY INTO @unload_stage/single_export/
FROM sales_load
FILE_FORMAT = (TYPE = 'CSV' HEADER = TRUE)
SINGLE = TRUE;
```

```
-- Unload to Parquet
COPY INTO @unload_stage/parquet_export/
FROM sales_load
FILE_FORMAT = (TYPE = 'PARQUET' SNAPPY_COMPRESSION = TRUE)
SINGLE = FALSE;

-- List exported files
LIST @unload_stage;

-- ============================================================
-- PART 8: Directory Tables
-- ============================================================
-- Enable directory table on the stage
ALTER STAGE unload_stage REFRESH;

-- Query directory table
SELECT * FROM DIRECTORY(@unload_stage);
-- Returns: relative_path, size, last_modified, file_url, md5

-- ============================================================
-- PART 9: Load History
-- ============================================================
USE ROLE ACCOUNTADMIN;

-- Review 64-day file load history for a table
SELECT *
FROM SNOWFLAKE.ACCOUNT_USAGE.LOAD_HISTORY
WHERE TABLE_NAME = 'SALES_LOAD'
  AND SCHEMA_NAME = 'STAGE_LAB'
  AND DATABASE_NAME = 'LOADING_LAB'
ORDER BY LAST_LOAD_TIME DESC
LIMIT 20;

-- Review copy history for a specific time range
SELECT *
FROM TABLE(INFORMATION_SCHEMA.COPY_HISTORY(
    TABLE_NAME => 'SALES_LOAD',
    START_TIME => DATEADD('day', -1, CURRENT_TIMESTAMP())
));
```

Topic 4.1 — 20 Practice Questions: Stages, File Formats & COPY INTO

Q1. How do you reference the User Stage in Snowflake SQL?

- A) `@user_stage`
- B) `@~`
- C) `@%username`
- D) `@$USER`

Q2. A developer wants to share staged files with multiple users and different teams. Which stage type is most appropriate?

- A) User Stage `@~` — visible to all users
- B) Table Stage `@%table` — accessible to all table users
- C) Named Internal Stage — a named schema-level object accessible to anyone with the appropriate USAGE and READ privileges
- D) External Stage connected to S3

Q3. A JSON file contains: `[{"id":1,"name":"Alice"},{"id":2,"name":"Bob"}]`. What file format parameter MUST be set to load each object as a separate row?

- A) `IGNORE_UTF8_ERRORS = TRUE`
- B) `STRIP_OUTER_ARRAY = TRUE`
- C) `STRIP_NULL_VALUES = TRUE`
- D) `ALLOW_DUPLICATE = TRUE`

Q4. What does the `ON_ERROR = SKIP_FILE` option do in a COPY INTO statement?

- A) Skips individual error rows within a file, continues loading remaining rows
- B) Skips the ENTIRE file that contains any errors, moves to the next file
- C) Stops the entire COPY operation at the first error in any file
- D) Skips files that already have been loaded in the past 64 days

Q5. A COPY INTO command is run with `FORCE = FALSE` (default). The same files were already loaded 30 days ago. What happens?

- A) The files are loaded again since 30 days < 64-day limit
- B) The files are skipped because they are still within the 64-day load history window
- C) Snowflake prompts the user to confirm whether to re-load the files
- D) The files are loaded only if the row count in the table has changed since last load

Q6. What does the `VALIDATION_MODE = RETURN_ERRORS` option in COPY INTO do?

- A) Loads the file and returns a count of error rows at the end
- B) Performs a dry run — validates the file and returns rows that WOULD fail, without actually loading any data
- C) Enables strict error mode where any error causes an immediate rollback
- D) Returns all rows from the file without any filtering or transformation

Q7. Which of the following correctly references a Table Stage for a table named `ORDERS`?

- A) `@orders`
- B) `@~ORDERS`
- C) `@%ORDERS`
- D) `@$ORDERS`

Q8. A Storage Integration is BEST used for which purpose?

- A) Storing file format definitions for reuse across multiple COPY INTO operations
- B) Creating a trusted, credential-free relationship between Snowflake and an external cloud storage provider
- C) Integrating Snowflake with third-party BI tools like Tableau
- D) Storing transformation logic to be applied during data loading

Q9. Which File Format types are natively supported in Snowflake? *(Select ALL that apply)*

- A) CSV
- B) JSON
- C) Parquet
- D) XML
- E) Avro
- F) ORC
- G) Excel (.xlsx)

Q10. A CSV file has a header row (column names). Which parameter ensures Snowflake skips it during load?

- A) `IGNORE_HEADER = TRUE`
- B) `HEADER = FALSE`
- C) `SKIP_HEADER = 1`
- D) `SKIP_ROWS = 1`

Q11. The `PURGE = TRUE` option in COPY INTO does what?

- A) Clears the COPY history for the files so they can be re-loaded later
- B) Deletes the source files from the stage after a successful load
- C) Purges all existing rows from the target table before loading
- D) Removes NULL values from loaded data before inserting into the table

Q12. A COPY INTO operation loads 3 files. File 2 has 50 error rows. `ON_ERROR = CONTINUE` is set. What is the outcome?

- A) The operation aborts when it hits the first error in File 2
- B) File 2 is skipped entirely; Files 1 and 3 are loaded successfully
- C) All valid rows from all 3 files are loaded; the 50 error rows in File 2 are skipped
- D) All 3 files are skipped because one file had errors

Q13. What does the `LIST @stage_name` command return?

- A) The schema definition (columns and data types) of the stage
- B) All files currently in the stage with their names, sizes, checksums, and last modified dates
- C) The load history of all files processed from the stage
- D) The access control list (ACL) for the stage

Q14. Which stage type does NOT support a Directory Table?

- A) Named Internal Stage
- B) Named External Stage
- C) User Stage (`@~`)
- D) Named External Stage with auto-refresh enabled

Q15. A developer wants to load a Parquet file where the column ORDER in the file differs from the target table's column order, but the column NAMES match. Which parameter should be used?

- A) `FORCE = TRUE`
- B) `MATCH_BY_COLUMN_NAME = CASE_INSENSITIVE`
- C) `COLUMN_ALIGNMENT = AUTO`
- D) `SKIP_HEADER = 1`

Q16. What is the maximum duration Snowflake tracks COPY INTO file load history per table for deduplication purposes?

- A) 7 days
- B) 14 days
- C) 30 days
- D) 64 days

Q17. When unloading data with `COPY INTO @stage FROM table` using `SINGLE = FALSE` (default), what is the result?

- A) A single compressed file is written to the stage
- B) Multiple parallel files are written to the stage — faster but no single output file
- C) Snowflake creates exactly 2 output files — one per warehouse cluster
- D) The unload fails because SINGLE must be specified explicitly

Q18. A Named External Stage is created using embedded AWS credentials. A few months later, the AWS credentials expire. Which approach avoids this problem in the future?

- A) Set a `CREDENTIAL_EXPIRY` parameter on the stage to auto-rotate every 90 days
- B) Use a Storage Integration instead of embedded credentials — the integration uses IAM roles that Snowflake manages

- C) Store the credentials in a Snowflake Secret object
- D) Set `AUTO_REFRESH_CREDENTIALS = TRUE` on the stage

Q19. What is the difference between `ON_ERROR = SKIP_FILE_5` and `ON_ERROR = SKIP_FILE_5%`?

- A) They are identical — both skip files with 5 errors
- B) `SKIP_FILE_5` skips a file if it has MORE THAN 5 error rows; `SKIP_FILE_5%` skips a file if the error RATE exceeds 5% of total rows
- C) `SKIP_FILE_5` skips 5 files; `SKIP_FILE_5%` skips files in 5% increments
- D) `SKIP_FILE_5` is deprecated; `SKIP_FILE_5%` is the current syntax

Q20. The `COPY_HISTORY` Information Schema function and `LOAD_HISTORY` Account Usage view both track file load history. What is the key difference?

- A) `COPY_HISTORY` shows only the last 7 days; `LOAD_HISTORY` shows 64 days
- B) `COPY_HISTORY` shows the last 14 days with lower latency (real-time); `LOAD_HISTORY` shows up to 365 days with 2-hour latency (Account Usage)
- C) `COPY_HISTORY` includes Snowpipe loads; `LOAD_HISTORY` only shows manual COPY INTO
- D) They are identical — just different names for the same data

SUBDOMAIN 4.2 — Automated Data Loading: Snowpipe, Streams & Tasks

Layman's Explanation

Manual COPY INTO is like hiring movers to show up once and move all the furniture. But what if furniture keeps arriving all day? You need automated movers.

Snowpipe = An automatic loader that watches a "drop zone" (stage) and loads new files as soon as they arrive — without you scheduling anything.

Streams = A motion-sensor on a table. When rows are added, updated, or deleted, the stream captures those changes so another process can pick them up.

Tasks = Scheduled jobs (like cron jobs). "Every 5 minutes, run this SQL." Combine with Streams: "Every 5 minutes, check for new changes in the stream and process them."

Together, Snowpipe + Streams + Tasks form the backbone of **real-time and near-real-time data pipelines** inside Snowflake.

Technical Explanation

Snowpipe — Continuous File-Based Ingestion[4][1][^3]

What it is: A serverless, managed service that automatically loads new files from a stage into a target table as soon as they arrive — without a running virtual warehouse.

Key characteristics:

- **Serverless:** No virtual warehouse needed — Snowflake manages the compute
- **Near-real-time:** Files typically loaded within seconds to minutes of arrival
- **Pay-per-use:** Billed based on serverless compute credits used during loading
- **NOT transactional:** Individual file loads may complete out of order — no ordering guarantee

Auto-Ingest (Event-Driven):[1][3]

- Uses cloud-native event notifications:
  - AWS: **SQS (Simple Queue Service)** notifies Snowpipe when new files land in S3
  - Azure: **Azure Event Grid** notifies when new blobs appear
  - GCS: **Pub/Sub** notifications
- Snowflake creates and manages the SQS queue — you configure your S3 bucket to send event notifications to it

```
-- Create a Snowpipe
CREATE OR REPLACE PIPE my_sales_pipe
    AUTO_INGEST = TRUE          -- Triggered by cloud events (SQS/Event Grid/Pub/Sub
    COMMENT = 'Auto-loads sales CSV files from S3'
AS
    COPY INTO sales_table
    FROM @my_external_stage
    FILE_FORMAT = (FORMAT_NAME = 'my_csv_format');

-- Get the SQS ARN to configure in S3 (for AUTO_INGEST = TRUE)
SHOW PIPES;
-- Look at notification_channel column — this is the SQS ARN for AWS

-- Manually trigger Snowpipe ingestion (REST API alternative to auto_ingest)
-- This is done via the Snowpipe REST API insertFiles endpoint — not SQL
```

Snowpipe load history:

- Available via `SNOWFLAKE.ACCOUNT_USAGE.PIPE_USAGE_HISTORY` (48-hour latency)
- Near-real-time: `SYSTEM$PIPE_STATUS()` — returns pipe status, pending files, credits used

```
-- Check Snowpipe status
SELECT SYSTEM$PIPE_STATUS('MY_SALES_PIPE');
-- Returns JSON: executionState, pendingFileCount, lastIngestedTimestamp
```

Snowpipe limitations (exam critical):

- Files loaded via Snowpipe also tracked by 64-day deduplication — won't re-load same file
- No ordering guarantee across files — use SEQUENCE or timestamps for ordering
- Default behavior: files must arrive AFTER the pipe is created to be auto-ingested
- Snowpipe uses COPY INTO internally — same file format and stage rules apply

Snowpipe Streaming (Row-Level Ingestion)[4][1]

What it is: A newer (COF-C03 feature) API-based ingestion method that allows streaming ROWS directly into Snowflake without staging files — using the **Snowflake Ingest SDK** or **Kafka Connector**.

Key differences from Snowpipe:

Feature	Snowpipe (File-based)	Snowpipe Streaming (Row-based)
Granularity	File-level (CSV, JSON, Parquet)	Row-level (individual records)
Latency	Seconds to minutes	1-5 seconds (very low)
Trigger	Cloud event notifications or REST API	SDK API call or Kafka Connector
Ordering	Not guaranteed across files	Sequence-based ordering available
Use case	Batch files landing in cloud storage	Real-time event streams, IoT, Kafka
Billing	Serverless compute per file loaded	Serverless compute per row

Streams — Change Data Capture (CDC)[4][3][^1]

What it is: A stream is a Snowflake object that records **data manipulation language (DML) changes** (INSERT, UPDATE, DELETE) made to a source table. It acts as a "cursor" — capturing changes since the last time the stream was consumed.

Stream types:

Type	Captures	Use Case
Standard (default)	INSERT, UPDATE, DELETE	Full CDC — complete change tracking
Append-Only	INSERT only (no UPDATE/DELETE)	Append-only ingestion pipelines — log tables, event tables
Insert-Only	INSERT only, works on EXTERNAL TABLES	Monitoring newly landed files in external tables

Stream columns (metadata columns):[3][1]

Column	Meaning
`METADATA$ACTION`	`INSERT` or `DELETE` (updates shown as DELETE + INSERT pair)
`METADATA$ISUPDATE`	`TRUE` if the row is part of an UPDATE (DELETE + INSERT where ISUPDATE = TRUE)
`METADATA$ROW_ID`	Unique ID for the row in the stream — stable across update cycles

How UPDATES appear in a Standard Stream:
An UPDATE to a row appears as TWO stream entries:

1. A `DELETE` with `METADATA$ISUPDATE = TRUE` (old values)
2. An `INSERT` with `METADATA$ISUPDATE = TRUE` (new values)

Stream staleness (critical edge case):

- Streams have a **staleness window** tied to the source table's Time Travel retention period (default 14 days for Standard edition, up to 90 days for Enterprise)
- If a stream is NOT consumed within the staleness window, it becomes STALE — and CANNOT be consumed anymore
- A stale stream must be RECREATED
- Check: `SHOW STREAMS` → look at `stale` column

```
-- Create streams
CREATE OR REPLACE STREAM orders_standard_stream
    ON TABLE orders
    COMMENT = 'Full CDC stream on orders table';

CREATE OR REPLACE STREAM orders_append_stream
    ON TABLE orders
    APPEND_ONLY = TRUE
    COMMENT = 'Append-only stream — only captures new rows';

-- Check if stream has data
SELECT SYSTEM$STREAM_HAS_DATA('ORDERS_STANDARD_STREAM') AS has_data;

-- Query stream contents
SELECT * FROM orders_standard_stream;
-- Returns rows with METADATA$ACTION, METADATA$ISUPDATE, METADATA$ROW_ID columns

-- Consuming a stream (DML consumes the stream records)
-- Important: consuming a stream in a transaction marks those records as processed
BEGIN TRANSACTION;
    INSERT INTO orders_processed
    SELECT order_id, customer_id, amount, METADATA$ACTION
    FROM orders_standard_stream
    WHERE METADATA$ACTION = 'INSERT' AND METADATA$ISUPDATE = FALSE;
COMMIT;
-- After commit: stream records just processed are removed from the stream
```

Tasks — Scheduled SQL Execution[1][4][^3]

What it is: A task is a scheduled Snowflake object that automatically executes a SQL statement (or stored procedure) on a defined schedule — like a cron job for Snowflake SQL.

Task types:

Type	Billing	When to Use
Serverless Task	Per compute second (serverless credits)	Variable workloads; Snowflake auto-sizes compute
Warehouse-Backed Task	Virtual warehouse credits	Consistent workloads; need control over warehouse size

Task scheduling:

```
-- CRON schedule (specific times)
SCHEDULE = 'USING CRON 0 */4 * * * UTC'    -- Every 4 hours
SCHEDULE = 'USING CRON 0 8 * * MON UTC'    -- Every Monday at 8 AM UTC
SCHEDULE = 'USING CRON */5 * * * * UTC'    -- Every 5 minutes

-- Minute interval (simpler)
SCHEDULE = '5 MINUTE'   -- Every 5 minutes
SCHEDULE = '1 HOUR'     -- Every hour
```

Task DAGs (Directed Acyclic Graphs):[4][3][^1]

- Tasks can be chained: a **root task** runs first, then **child tasks** run after the root completes
- Only the ROOT task has a schedule — child tasks are triggered by parent completion
- A task DAG can have max **1,000 tasks**
- **Finalizer task:** A special task that always runs after the root task's DAG completes — even if errors occurred. Used for cleanup, notifications.

```
-- Root task (runs on schedule)
CREATE OR REPLACE TASK root_load_task
    WAREHOUSE = 'load_wh'
    SCHEDULE = '5 MINUTE'
    -- or: USER_TASK_MANAGED_INITIAL_WAREHOUSE_SIZE = 'XSMALL' for serverless
    WHEN SYSTEM$STREAM_HAS_DATA('ORDERS_STANDARD_STREAM')  -- Only run if stream has d
AS
    INSERT INTO orders_staging
    SELECT order_id, customer_id, amount, METADATA$ACTION
    FROM orders_standard_stream
    WHERE METADATA$ACTION = 'INSERT';

-- Child task (triggered after root_load_task completes)
CREATE OR REPLACE TASK transform_task
    WAREHOUSE = 'load_wh'
    AFTER root_load_task                -- This is a child of root_load_task
AS
    UPDATE orders_final
```

```
    SET processed = TRUE
    WHERE order_id IN (SELECT order_id FROM orders_staging WHERE processed = FALSE);

-- Finalizer task (runs after entire DAG, even on failure)
CREATE OR REPLACE TASK cleanup_task
    WAREHOUSE = 'load_wh'
    FINALIZE = root_load_task            -- Finalizer for this root task's DAG
AS
    DELETE FROM orders_staging WHERE inserted_at < DATEADD('hour', -24, CURRENT_TIMES

-- Tasks are SUSPENDED by default when created — must be RESUMED
ALTER TASK transform_task RESUME;
ALTER TASK cleanup_task RESUME;
ALTER TASK root_load_task RESUME;        -- Resume ROOT task last

-- Suspend all tasks
ALTER TASK root_load_task SUSPEND;
```

`WHEN` clause (task conditional execution):

- `WHEN SYSTEM$STREAM_HAS_DATA('stream_name')` — run only if stream has unprocessed changes
- Tasks with a WHEN condition that evaluates to FALSE are "skipped" — but they're not billed for serverless tasks

`SUSPEND_TASK_AFTER_NUM_FAILURES`:

- Default: 10 (task auto-suspends after 10 consecutive failures)
- Set to 0 to disable auto-suspension (task runs forever even on repeated failures)

Dynamic Tables — Declarative Pipelines[3][1][^4]

What it is: A newer feature that lets you define a table as a SQL query — Snowflake automatically and continuously refreshes the table's contents based on source table changes. It's like a materialized view with automated incremental refresh.

KEY difference from Materialized Views:

Feature	Materialized View	Dynamic Table
Purpose	Query acceleration (faster reads)	Data pipeline (automated transformation)
Refresh	Automatic, incremental (serverless)	Automatic, based on TARGET_LAG
BASE	Pre-computed query result	Result of an arbitrary SQL query
Joins	Limited	Fully supports complex queries
Window functions	Limited	Fully supported
Nesting	Not supported	Can chain Dynamic Tables
Use case	Fast reads of aggregated data	ELT pipelines, replacing streams + tasks

TARGET_LAG: How far behind the Dynamic Table can be from source data.

- `TARGET_LAG = '1 minute'` — refresh within 1 minute of source changes
- `TARGET_LAG = DOWNSTREAM` — only refresh when a downstream consumer (another Dynamic Table) needs it

Hands-On Queries: Snowpipe, Streams & Tasks

```
-- ============================================================
-- SETUP
-- ============================================================
USE ROLE SYSADMIN;
USE DATABASE LOADING_LAB;
CREATE SCHEMA IF NOT EXISTS PIPELINE_LAB;
USE SCHEMA PIPELINE_LAB;
USE WAREHOUSE load_wh;

-- Source table
CREATE OR REPLACE TABLE raw_orders (
    order_id    INTEGER,
    customer_id INTEGER,
    order_date  DATE,
    amount      NUMBER(12,2),
    status      VARCHAR(20),
    region      VARCHAR(30)
);

-- Processed / destination tables
CREATE OR REPLACE TABLE orders_processed (
    order_id    INTEGER,
    customer_id INTEGER,
    order_date  DATE,
    amount      NUMBER(12,2),
    status      VARCHAR(20),
    region      VARCHAR(30),
    processed_at TIMESTAMP_LTZ DEFAULT CURRENT_TIMESTAMP(),
    cdc_action   VARCHAR(10)
);

-- ============================================================
-- PART 1: Streams
-- ============================================================
-- Create a standard (full CDC) stream
CREATE OR REPLACE STREAM orders_cdc_stream
    ON TABLE raw_orders
    COMMENT = 'Full CDC stream — captures INSERTs, UPDATEs, and DELETEs';

-- Create an append-only stream
CREATE OR REPLACE STREAM orders_append_stream
    ON TABLE raw_orders
    APPEND_ONLY = TRUE
    COMMENT = 'Append-only — only captures new INSERTs';
```

```
-- Check if stream has data
SELECT SYSTEM$STREAM_HAS_DATA('ORDERS_CDC_STREAM') AS cdc_has_data,
       SYSTEM$STREAM_HAS_DATA('ORDERS_APPEND_STREAM') AS append_has_data;

-- Insert rows into source table → this will populate the streams
INSERT INTO raw_orders VALUES
    (1001, 5001, '2025-01-15', 250.00, 'PENDING',   'North'),
    (1002, 5002, '2025-01-15', 899.50, 'PENDING',   'South'),
    (1003, 5003, '2025-01-16', 125.75, 'PENDING',   'East'),
    (1004, 5004, '2025-01-16', 3400.00,'PENDING',   'West');

-- Check stream — should now have data
SELECT SYSTEM$STREAM_HAS_DATA('ORDERS_CDC_STREAM') AS has_data;
SELECT * FROM orders_cdc_stream;
-- See rows with METADATA$ACTION='INSERT', METADATA$ISUPDATE=FALSE

-- Now perform an UPDATE — watch how stream captures it
UPDATE raw_orders SET status = 'APPROVED' WHERE order_id = 1001;

-- Check CDC stream (should see DELETE + INSERT pair for order 1001)
SELECT order_id, status, METADATA$ACTION, METADATA$ISUPDATE
FROM orders_cdc_stream
ORDER BY order_id, METADATA$ACTION;
-- order_id 1001: appears TWICE
--   Row 1: METADATA$ACTION='DELETE', METADATA$ISUPDATE=TRUE (old PENDING value)
--   Row 2: METADATA$ACTION='INSERT', METADATA$ISUPDATE=TRUE (new APPROVED value)
-- order_ids 1002-1004: METADATA$ACTION='INSERT', METADATA$ISUPDATE=FALSE (original i

-- Append-only stream: Should only show the original INSERT rows (NOT the update)
SELECT order_id, status, METADATA$ACTION FROM orders_append_stream;
-- Only shows the 4 original INSERTs — the UPDATE to 1001 is NOT captured

-- Perform a DELETE
DELETE FROM raw_orders WHERE order_id = 1004;

-- CDC stream now has the DELETE record
SELECT order_id, status, METADATA$ACTION FROM orders_cdc_stream;

-- Consume the CDC stream (DML consumes the stream changes)
BEGIN TRANSACTION;
    INSERT INTO orders_processed (order_id, customer_id, order_date, amount, status
    SELECT order_id, customer_id, order_date, amount, status, region, METADATA$ACTI
    FROM orders_cdc_stream;
COMMIT;

-- After consumption: stream is empty
SELECT SYSTEM$STREAM_HAS_DATA('ORDERS_CDC_STREAM') AS has_data;  -- FALSE

SELECT * FROM orders_processed;  -- See all CDC events including UPDATE and DELETE

-- Stream metadata
SHOW STREAMS IN SCHEMA PIPELINE_LAB;
-- Look for: stale column, stale_after column, source_object

-- ============================================================
-- PART 2: Tasks
```

```
-- ============================================================
-- Create a new staging table for task demo
CREATE OR REPLACE TABLE orders_staging (
    order_id   INTEGER,
    amount     NUMBER(12,2),
    cdc_action VARCHAR(10),
    staged_at  TIMESTAMP_LTZ DEFAULT CURRENT_TIMESTAMP()
);

-- Root task: Runs every 2 minutes if stream has data
CREATE OR REPLACE TASK process_new_orders_task
    WAREHOUSE = load_wh
    SCHEDULE  = '2 MINUTE'
    WHEN      SYSTEM$STREAM_HAS_DATA('ORDERS_APPEND_STREAM')
AS
    INSERT INTO orders_staging (order_id, amount, cdc_action)
    SELECT order_id, amount, METADATA$ACTION
    FROM orders_append_stream;

-- Child task: Triggered after root task completes
CREATE OR REPLACE TASK archive_staged_task
    WAREHOUSE = load_wh
    AFTER process_new_orders_task
AS
    DELETE FROM orders_staging
    WHERE staged_at < DATEADD('hour', -48, CURRENT_TIMESTAMP());

-- Tasks are SUSPENDED by default — resume CHILDREN first, then ROOT
ALTER TASK archive_staged_task RESUME;
ALTER TASK process_new_orders_task RESUME;

-- Verify task status
SHOW TASKS IN SCHEMA PIPELINE_LAB;
-- state column: 'started' (scheduled), 'suspended'

-- Add new rows to trigger the task on next schedule run
INSERT INTO raw_orders VALUES
    (1005, 5005, CURRENT_DATE(), 500.00, 'PENDING', 'North'),
    (1006, 5006, CURRENT_DATE(), 750.00, PENDING',  'East');

-- Manually execute task immediately (bypasses schedule — for testing)
EXECUTE TASK process_new_orders_task;

-- Check task run history
SELECT *
FROM TABLE(INFORMATION_SCHEMA.TASK_HISTORY(
    SCHEDULED_TIME_RANGE_START => DATEADD('hour', -1, CURRENT_TIMESTAMP()),
    TASK_NAME => 'PROCESS_NEW_ORDERS_TASK'
))
ORDER BY SCHEDULED_TIME DESC;
-- Shows: SCHEDULED_TIME, QUERY_START_TIME, COMPLETED_TIME, STATE (succeeded/failed/s

-- Account Usage task history (longer retention)
USE ROLE ACCOUNTADMIN;
SELECT *
FROM SNOWFLAKE.ACCOUNT_USAGE.TASK_HISTORY
```

```
WHERE SCHEDULED_TIME >= DATEADD('day', -7, CURRENT_TIMESTAMP())
  AND DATABASE_NAME = 'LOADING_LAB'
ORDER BY SCHEDULED_TIME DESC
LIMIT 30;

-- Suspend tasks when done
USE ROLE SYSADMIN;
ALTER TASK process_new_orders_task SUSPEND;
ALTER TASK archive_staged_task SUSPEND;

-- ============================================================
-- PART 3: Dynamic Tables
-- ============================================================
-- Create a dynamic table that auto-refreshes from raw_orders
CREATE OR REPLACE DYNAMIC TABLE regional_order_summary
    TARGET_LAG = '1 minute'           -- Refresh within 1 minute of source changes
    WAREHOUSE = load_wh
    AS
    SELECT
        region,
        DATE_TRUNC('day', order_date)  AS order_day,
        COUNT(*)                      AS num_orders,
        SUM(amount)                    AS total_amount,
        AVG(amount)                   AS avg_amount,
        MAX(amount)                   AS max_amount
    FROM raw_orders
    WHERE status != 'CANCELLED'
    GROUP BY region, order_day;

-- Check dynamic table status
SHOW DYNAMIC TABLES IN SCHEMA PIPELINE_LAB;
-- Look for: target_lag, scheduling_state, data_timestamp columns

-- Query the dynamic table (auto-refreshed from source)
SELECT * FROM regional_order_summary ORDER BY region, order_day;

-- Add new rows to raw_orders — dynamic table will refresh automatically
INSERT INTO raw_orders VALUES
    (1007, 5007, CURRENT_DATE(), 999.00, 'PENDING', 'North'),
    (1008, 5008, CURRENT_DATE(), 1200.00, 'PENDING', 'West');

-- Wait ~1 minute, then re-query — should reflect new totals
SELECT * FROM regional_order_summary ORDER BY region;

-- Manually trigger a refresh (bypasses lag)
ALTER DYNAMIC TABLE regional_order_summary REFRESH;

-- Check refresh history
SELECT *
FROM INFORMATION_SCHEMA.DYNAMIC_TABLE_REFRESH_HISTORY(
    NAME => 'REGIONAL_ORDER_SUMMARY'
)
ORDER BY DATA_TIMESTAMP DESC
LIMIT 10;

-- ============================================================
```

```
-- PART 4: Snowpipe Status (demo without actual S3)
-- ============================================================
-- Create a demo internal stage pipe
CREATE OR REPLACE STAGE pipe_demo_stage;

-- Create table for pipe
CREATE OR REPLACE TABLE pipe_target (data VARIANT);

-- Create a Snowpipe (without AUTO_INGEST for internal stage demo)
CREATE OR REPLACE PIPE internal_demo_pipe
    AUTO_INGEST = FALSE   -- No cloud event notification needed for internal stage
AS
    COPY INTO pipe_target
    FROM @pipe_demo_stage
    FILE_FORMAT = (TYPE = 'JSON' STRIP_OUTER_ARRAY = TRUE);

-- Check pipe status
SELECT SYSTEM$PIPE_STATUS('INTERNAL_DEMO_PIPE');

-- Pause and resume a pipe
ALTER PIPE internal_demo_pipe PAUSE;
ALTER PIPE internal_demo_pipe RESUME;

-- Show all pipes
SHOW PIPES IN SCHEMA PIPELINE_LAB;

-- Cleanup
DROP TABLE IF EXISTS pipe_target;
DROP STAGE IF EXISTS pipe_demo_stage;
DROP PIPE IF EXISTS internal_demo_pipe;
DROP DYNAMIC TABLE IF EXISTS regional_order_summary;
DROP TASK IF EXISTS archive_staged_task;
DROP TASK IF EXISTS process_new_orders_task;
DROP STREAM IF EXISTS orders_cdc_stream;
DROP STREAM IF EXISTS orders_append_stream;
DROP DATABASE IF EXISTS LOADING_LAB;
```

Topic 4.2 — 20 Practice Questions: Snowpipe, Streams & Tasks

Q1. What makes Snowpipe different from a manually-run COPY INTO statement?

- A) Snowpipe can load JSON files; COPY INTO can only load CSV
- B) Snowpipe is serverless and continuously monitors a stage for new files, loading them automatically without a running virtual warehouse
- C) Snowpipe loads data faster because it uses a larger internal warehouse
- D) Snowpipe can only be used with external stages; COPY INTO works with internal stages

Q2. A Snowpipe is configured with `AUTO_INGEST = TRUE` against an S3 bucket. What AWS service is used to trigger Snowpipe when new files arrive?

- A) AWS Lambda

- B) AWS CloudWatch Events
- C) AWS SQS (Simple Queue Service)
- D) AWS Kinesis Firehose

Q3. An `UPDATE` is performed on a row in a table being tracked by a Standard Stream. How does the UPDATE appear in the stream?

- A) As a single row with `METADATA$ACTION = 'UPDATE'`
- B) As two rows: a DELETE (old values, METADATA $ISUPDATE = TRUE) and an INSERT (new values, METADATA ISUPDATE=TRUE)
- C) The UPDATE is not captured — standard streams only track INSERT and DELETE
- D) As one row with `METADATA$ACTION = 'MODIFY'`

Q4. What is the correct order to resume a task DAG with a root task and child tasks?

- A) Resume the ROOT task first, then child tasks
- B) Resume CHILD tasks first, then the ROOT task
- C) All tasks in a DAG must be resumed simultaneously with a single command
- D) Tasks are automatically resumed when data arrives — no manual resume needed

Q5. A stream on table `ORDERS` hasn't been consumed in 20 days, and the table has the default Time Travel retention of 1 day (Standard edition). What happened to the stream?

- A) The stream is still valid — streams have a 90-day retention period independent of Time Travel
- B) The stream became STALE because the staleness window (tied to Time Travel retention) elapsed without consumption
- C) The stream was automatically purged by Snowflake after 14 days
- D) The stream is still valid but paused — it will resume on next consumption

Q6. An Append-Only stream is created on an `ORDERS` table. What operations does it capture?

- A) INSERT, UPDATE, and DELETE operations (all DML)
- B) INSERT operations only — UPDATE and DELETE changes are not recorded
- C) UPDATE and DELETE only (not INSERT, since those are not changes)
- D) Only bulk COPY INTO operations — not individual row inserts

Q7. What does `WHEN SYSTEM$STREAM_HAS_DATA('my_stream')` in a Task definition do?

- A) Runs the task every time the stream count increases by 1
- B) Makes the task check if the stream has unconsumed data before running — if no data, the task is SKIPPED without executing (and without consuming credits for serverless tasks)

- C) Locks the stream while the task is running to prevent concurrent writes
- D) Creates an automatic dependency between the task and the stream

Q8. What is the MAXIMUM number of tasks allowed in a single Task DAG?

- A) 100
- B) 500
- C) 1,000
- D) Unlimited

Q9. What is a Finalizer Task in Snowflake?

- A) A task that runs BEFORE the root task to validate data prerequisites
- B) A special task that always executes AFTER the entire root task DAG completes — even if some tasks in the DAG failed — used for cleanup and notifications
- C) The last child task in a linear chain (not parallel)
- D) A task that marks all stream records as "finalized" (non-consumable) after processing

Q10. Snowpipe Streaming differs from regular Snowpipe in which key way?

- A) Snowpipe Streaming is free; regular Snowpipe charges serverless credits
- B) Snowpipe Streaming is file-based; regular Snowpipe is row-based
- C) Snowpipe Streaming uses a row-level SDK/Kafka integration to ingest individual records with 1-5 second latency; regular Snowpipe loads files
- D) Snowpipe Streaming requires a dedicated virtual warehouse; regular Snowpipe is serverless

Q11. A developer consumes records from a stream inside a DML statement that is ROLLED BACK. What happens to the stream records?

- A) The records are permanently consumed — rollback does not restore stream records
- B) The records are returned to the stream — a rollback reverts the stream consumption
- C) The records are moved to a dead-letter queue for manual review
- D) The stream is automatically suspended after a rollback

Q12. What does `SUSPEND_TASK_AFTER_NUM_FAILURES = 0` mean on a task?

- A) The task is immediately suspended before it runs for the first time
- B) The task will NEVER automatically suspend due to failures — it will keep retrying regardless of how many times it fails
- C) The task suspends after 0 consecutive successes (never runs)
- D) The task suspends immediately after any single failure

Q13. Which of the following would be the BEST approach for loading files that land in an S3 bucket every 15-30 minutes throughout the day?

- A) Schedule a Task every 15 minutes to run COPY INTO from the S3 stage
- B) Use Snowpipe with `AUTO_INGEST = TRUE` — it triggers automatically when files land in S3 via SQS events, eliminating poll-wait delays
- C) Create a Stream on the external stage and consume it every hour
- D) Use the COPY INTO command with FORCE = TRUE every 15 minutes

Q14. What is `TARGET_LAG` in a Dynamic Table?

- A) The maximum allowed time difference between the Dynamic Table's data and the current system time
- B) The maximum acceptable time by which the Dynamic Table can lag behind its source tables before Snowflake triggers a refresh
- C) The scheduled refresh interval — the Dynamic Table refreshes exactly at this interval
- D) The number of seconds it takes to query the Dynamic Table

Q15. What is the key ARCHITECTURAL difference between a Dynamic Table and a Stream + Task combination?

- A) They are functionally identical — just different syntax for the same operation
- B) Dynamic Tables use a DECLARATIVE approach (define the desired result; Snowflake manages refresh automatically); Stream + Task uses an IMPERATIVE approach (manually define how and when changes are processed)
- C) Dynamic Tables only support batch refresh; Stream + Task enables real-time processing
- D) Dynamic Tables require Enterprise edition; Stream + Task is available in all editions

Q16. A Standard Stream on table `ORDERS` is read by User A at 9 AM but the consuming DML is NOT committed (session is left open). User B queries the same stream at 10 AM. What does User B see?

- A) User B sees an empty stream — User A's read already consumed the records
- B) User B sees all stream records because the stream is only consumed upon DML COMMIT, not just reading
- C) User B is blocked — only one session can access a stream at a time
- D) User B sees a copy of the stream with only the records User A didn't read

Q17. Which of the following statements about Snowpipe are TRUE? *(Select ALL that apply)*

- A) Snowpipe is serverless — no virtual warehouse is required
- B) Snowpipe files are also tracked by the 64-day deduplication history
- C) Snowpipe guarantees delivery order — files are loaded in the exact order they arrive

- D) Snowpipe's notification channel (SQS ARN) can be found via `SHOW PIPES`
- E) Snowpipe loads are visible in `SNOWFLAKE.ACCOUNT_USAGE.PIPE_USAGE_HISTORY`

Q18. Which stream type would you use to monitor an External Table for newly arrived files in an external cloud storage location?

- A) Standard Stream — captures all changes including new file arrivals
- B) Append-Only Stream — captures newly inserted rows in the external table
- C) Insert-Only Stream — specifically designed for external tables where only new file arrivals (inserts) occur
- D) Directory Table — not a stream type but appropriate for monitoring file arrivals

Q19. What is the difference between a Root Task and a Child Task in a task DAG?

- A) Root Tasks execute SQL; Child Tasks execute Stored Procedures only
- B) Only the ROOT task has a SCHEDULE; CHILD tasks are triggered by parent task completion (using the AFTER parameter). Only Root tasks can have a WHEN condition.
- C) Root Tasks are serverless; Child Tasks require a virtual warehouse
- D) Child Tasks run in parallel with the Root Task — not after it

Q20. When using `EXECUTE TASK` to manually run a task, what happens to the scheduled run?

- A) The manual run cancels the next scheduled run
- B) The manual run executes the task immediately as a one-time run; the regular schedule continues independently — the next scheduled run still fires as planned
- C) `EXECUTE TASK` can only be used on suspended tasks — it fails if the task is active
- D) The manual run resets the schedule timer, delaying the next scheduled run

✅ COMPLETE ANSWER KEY — Domain 4: Data Loading, Unloading & Connectivity

Topic 4.1 — Stages, File Formats & COPY INTO

Q	Answer	Key Reason
1	B	User Stage is always referenced as `@~` — the tilde is the shorthand for the current user's stage.[1][3]
2	C	Named Internal Stage is a schema-level object that can be shared via privileges across roles and users — unlike User Stages (per user) or Table Stages (per table).[1][3]
3	B	`STRIP_OUTER_ARRAY = TRUE` removes the `[...]` wrapper from a JSON array of objects so each object becomes a separate row, rather than loading the entire array as one VARIANT.[1][3]

Q	Answer	Key Reason
4	B	`ON_ERROR = SKIP_FILE` skips the entire file containing any error. The operation continues to the next file.[1][3]
5	B	The 64-day load history deduplication prevents the same files from being re-loaded within 64 days. The files are SKIPPED since they were loaded 30 days ago (within the window).[1][3]
6	B	`VALIDATION_MODE = RETURN_ERRORS` performs a dry run — validates the file, returns rows that would fail, WITHOUT loading any data into the table.[1][3]
7	C	Table Stage is referenced as `@%table_name`. For a table named ORDERS: `@%ORDERS`.[1][3]
8	B	Storage Integration creates a trusted IAM-based relationship between Snowflake and external cloud storage — eliminating the need to embed and manage credentials.[4][3]
9	A, B, C, D, E, F	CSV, JSON, Parquet, XML, Avro, and ORC are all natively supported Snowflake file formats. Excel (.xlsx) is NOT natively supported.[1][3]
10	C	`SKIP_HEADER = 1` tells COPY INTO to skip the first N lines of each file (the header row).[1][3]
11	B	`PURGE = TRUE` deletes the source files from the stage after a successful COPY INTO load. Does NOT affect the target table data or load history.[1][3]
12	C	`ON_ERROR = CONTINUE` skips individual error rows but loads all valid rows from ALL files. The 50 error rows in File 2 are skipped; their valid rows and all of Files 1 and 3 are loaded.[1][3]
13	B	`LIST @stage` returns all files in the stage with their names, sizes (bytes), MD5 checksums, and last modified timestamps.[^3]
14	C	User Stage (@~) does NOT support Directory Tables. Directory Tables are only available on Named Stages (internal and external).[4][3]
15	B	`MATCH_BY_COLUMN_NAME = CASE_INSENSITIVE` (or CASE_SENSITIVE) matches file columns to table columns by NAME instead of position — essential for Parquet/ORC/Avro where column order may differ.[1][3]
16	D	Snowflake tracks COPY INTO file load history per table for 64 days for deduplication.[1][3]
17	B	`SINGLE = FALSE` (default) writes multiple parallel output files — faster but produces multiple files. `SINGLE = TRUE` produces one file but is slower (no parallelism).[1][3]
18	B	Storage Integration uses Snowflake's IAM identity (ARN + external ID) to authenticate — no access keys to expire. Embedded credentials expire and require manual stage updates.[4][3]
19	B	`SKIP_FILE_5` = skip file if MORE THAN 5 rows have errors (absolute count). `SKIP_FILE_5%` = skip file if MORE THAN 5% of rows have errors (percentage rate).[1][3]
20	B	`INFORMATION_SCHEMA.COPY_HISTORY` shows last 14 days with low latency (near-real-time). `ACCOUNT_USAGE.LOAD_HISTORY` shows up to 365 days but has 2-hour latency.[1][3]

Topic 4.2 — Snowpipe, Streams & Tasks

Q	Answer	Key Reason
1	B	Snowpipe is serverless and automatically loads files as they arrive in a stage — no manual trigger, no running warehouse required.[1][4]

Q	Answer	Key Reason
2	C	AWS SQS (Simple Queue Service) is used to deliver event notifications from S3 to Snowpipe when new files land. Snowflake creates and manages the SQS queue.[1][3]
3	B	A Standard Stream represents UPDATEs as two rows: a DELETE of the old values (METADATA$ISUPDATE=TRUE) + an INSERT of the new values (METADATA$ISUPDATE=TRUE). [1][3]
4	B	Always resume CHILD tasks first, then the ROOT task. If you resume the root first, it may fire before the children are active, causing the DAG to fail.[1][4][^3]
5	B	Stream staleness is tied to the source table's Time Travel retention (default 1 day on Standard edition). After 20 days without consumption, the stream became STALE and must be recreated.[1][3]
6	B	Append-Only streams capture INSERT operations only. UPDATE and DELETE operations are NOT recorded in an Append-Only stream.[1][3]
7	B	The WHEN clause evaluates the condition before running the task. If SYSTEM$STREAM_HAS_DATA returns FALSE, the task is SKIPPED — saving credits on serverless tasks and avoiding unnecessary runs.[1][4][^3]
8	C	A single Task DAG supports a maximum of 1,000 tasks.[1][4]
9	B	A Finalizer Task always runs after the root task's DAG completes — even if some tasks in the DAG failed. Used for cleanup, notifications, and guaranteed post-processing.[1][4][^3]
10	C	Snowpipe Streaming uses a row-level SDK/Kafka integration for 1-5 second latency; regular Snowpipe loads files. Regular Snowpipe = file-based; Snowpipe Streaming = row-based.[1][4]
11	B	Stream consumption only completes on DML COMMIT. If the transaction is rolled back, the records return to the stream — they can be consumed again by the next run.[1][3]
12	B	SUSPEND_TASK_AFTER_NUM_FAILURES = 0 disables automatic suspension — the task never auto-suspends due to failures regardless of how many consecutive failures occur.[1][4]
13	B	Snowpipe with AUTO_INGEST = TRUE is the best choice — it reacts to file arrival events via SQS immediately, with no polling delay. A scheduled Task would have up to 15-minute lag.[1][3]
14	B	TARGET_LAG specifies how far behind the Dynamic Table can lag from its source. Snowflake triggers a refresh to keep the Dynamic Table within this lag window.[1][4][^3]
15	B	Dynamic Tables = DECLARATIVE (define what the result should be; Snowflake figures out how and when to refresh). Stream + Task = IMPERATIVE (you write the logic and control the schedule). [1][4][^3]
16	B	Streams are consumed by DML COMMIT — not by reading. User A's uncommitted SELECT doesn't consume the records. User B sees all the same records.[1][3]
17	A, B, D, E	Serverless ✓; 64-day dedup ✓; SQS ARN in SHOW PIPES ✓; PIPE_USAGE_HISTORY ✓. Delivery ORDER is NOT guaranteed ✗ — files may load out of order.[1][4][^3]
18	C	Insert-Only Stream is specifically designed for External Tables — external tables don't support UPDATE/DELETE, so only new file arrivals (inserts) are tracked.[1][3]
19	B	Only the ROOT task has a SCHEDULE. CHILD tasks use the AFTER parameter to declare their parent dependency and are triggered automatically after parent completion.[1][4][^3]
20	B	EXECUTE TASK is an immediate one-time execution. The regular scheduled runs continue independently — the schedule is not affected or reset.[1][4][^3]

References

1. Snow Pro Core Study Guide C 03 | PDF | Databases - Scribd - The SnowPro Core Exam Study Guide provides an overview of the exam content, format, and preparation ...
2. SnowPro Core Exam Prep 2026 - App Store - Apple - Pass the Snowflake SnowPro Core exam with confidence. 2,100+ questions, progressive difficulty, and ...
3. Free Practice Questions for Snowflake COF-C03 Certification - Study with 563 exam-style practice questions designed to help you prepare for the Snowflake SnowPro ...
4. SnowPro® Core Certification (COF-C03) - Snowflake University - Access the Exam Study Guide. This guide provides information that will help you prepare for your cer...